

[LOGO DE L'ÉCOLE / UNIVERSITÉ — À AJOUTER]
SMARTOVATE LTD — À AJOUTER]

[LOGO

RAPPORT DE STAGE

Stage de fin d'études — 2 mois (Juillet – Août 2026)

Agent IA Autonome avec AWS Bedrock + LangChain

Conception, développement et déploiement d'un agent conversationnel autonome de type ReAct, orchestré avec LangGraph, déployé en architecture serverless sur AWS

Présenté par :
SIMPORE W. Flore

Encadrement :
Maître de stage (Smartovate Ltd) : [À
COMPLÉTER]
Tuteur académique : [À COMPLÉTER]

Entreprise d'accueil : Smartovate Ltd
[À COMPLÉTER : ville / pays du siège Smartovate]

Période de stage : 1^{er} juillet – 25 août 2026

[À COMPLÉTER : nom de l'établissement, filière, année académique]

Remerciements

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Section à personnaliser entièrement : remerciements au maître de stage, au tuteur académique, à l'équipe Smartovate Ltd, à l'établissement, et à toute personne ayant contribué à ce stage. Le paragraphe ci-dessous n'est qu'un squelette à adapter.

Je tiens à exprimer ma sincère gratitude à l'ensemble de l'équipe de Smartovate Ltd pour son accueil et son accompagnement tout au long de ce stage de deux mois. Je remercie tout particulièrement **[nom du maître de stage]**, pour sa disponibilité, ses conseils techniques et la confiance accordée sur ce projet d'agent IA autonome. Je remercie également **[nom du tuteur académique]** pour son suivi pédagogique, ainsi que l'ensemble du corps enseignant de **[établissement]** pour la formation qui a rendu ce travail possible.

Résumé

Ce rapport présente le travail réalisé durant un stage de deux mois au sein de Smartovate Ltd, entreprise de conseil cloud, portant sur la conception et le développement d'un agent d'intelligence artificielle autonome. Le projet combine **AWS Bedrock** (accès managé aux modèles de fondation Claude et Titan) et **LangChain** / **LangGraph** (orchestration) pour produire un agent suivant le patron **ReAct** (Reasoning + Acting), capable de mobiliser cinq outils (calculatrice, météo, recherche web, recherche documentaire interne, recherche dans les documents utilisateur), de maintenir une mémoire conversationnelle court terme et une mémoire sémantique long terme (OpenSearch Serverless), et d'être interrogé via une interface Streamlit ou une API HTTP streamée (Server-Sent Events).

Au-delà du développement fonctionnel, une part importante du stage a consisté à **fiabiliser** le système existant : mise en place d'une suite de 161 tests unitaires (logique des cinq outils, cœur de la boucle ReAct, persistance, limitation de débit), construction d'un harnais d'évaluation de bout en bout sur 23 scénarios représentatifs (95,7 % de réussite en conditions réelles), correction de plusieurs anomalies fonctionnelles et de sécurité identifiées en cours de route (authentification silencieuse défaillante, mémoire sémantique jamais persistée, permissions IAM trop permissives, secret d'API en clair, garde-fous de sécurité invisibles côté client, encodage des caractères corrompu), et durcissement de l'infrastructure définie en AWS CDK selon le principe du moindre privilège.

Mots-clés : intelligence artificielle générative, agent autonome, ReAct, LangChain, LangGraph, AWS Bedrock, AWS Lambda, architecture serverless, retrieval-augmented generation (RAG), tests logiciels.

Abstract

This report presents the work carried out during a two-month internship at Smartovate Ltd, a cloud consulting company, focused on designing and developing an autonomous artificial intelligence agent. The project combines **AWS Bedrock** (managed access to Claude and Titan foundation models) with **LangChain / LangGraph** (orchestration) to build an agent following the **ReAct** (Reasoning + Acting) pattern, capable of using five tools (calculator, weather, web search, internal knowledge-base retrieval, and user-document retrieval), maintaining both short-term conversational memory and long-term semantic memory (OpenSearch Serverless), and being queried through a Streamlit interface or a streamed HTTP API (Server-Sent Events).

Beyond feature development, a significant part of the internship consisted of **hardening** the existing system: building a suite of 161 unit tests, an end-to-end evaluation harness covering 23 representative scenarios (95.7% success rate in live conditions), fixing several functional and security issues discovered along the way, and tightening the AWS CDK infrastructure to follow the least-privilege principle.

Keywords: generative artificial intelligence, autonomous agent, ReAct, LangChain, LangGraph, AWS Bedrock, AWS Lambda, serverless architecture, retrieval-augmented generation (RAG), software testing.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Table des figures	v
Liste des tableaux	v
Liste des abréviations	1
1 Introduction	2
1.1 Contexte et enjeux	2
1.2 Objectifs du stage	2
1.3 Organisation du rapport	3

2	Présentation de l'entreprise	4
2.1	Activité	4
2.2	Organisation de l'équipe d'accueil	4
3	Cadrage du projet	5
3.1	Périmètre	5
3.2	Stack technologique visée	5
3.3	Planning prévisionnel	6
4	Méthodologie de travail	7
4.1	Approche Agile et gestion de projet	7
4.2	Suivi via Jira	7
5	Conception de la solution	9
5.1	Vue d'ensemble de l'architecture	9
5.2	Composants principaux	10
5.3	Modèle de données	10
5.4	Le patron ReAct et son orchestration par LangGraph	10
5.5	Les cinq outils de l'agent	12
5.6	Sécurité	12
6	Réalisation	13
6.1	Epic 1 — Setup AWS & Configuration Bedrock	13
6.2	Epic 2 — Développement de l'agent et de ses outils	13
6.3	Epic 3 — Mémoire et persistance	14
6.4	Epic 4 — Déploiement serverless et API	14
7	Tests et validation	15
7.1	Tests unitaires	15
7.2	Harnais d'évaluation de bout en bout	16
8	Difficultés rencontrées et solutions	17
8.1	Reconnexion silencieuse défaillante	17
8.2	Mémoire sémantique jamais indexée	17
8.3	Permissions IAM excessivement larges	17
8.4	Secret d'API en clair dans le terminal	18
8.5	Garde-fous de sécurité invisibles côté utilisateur	18
8.6	Encodage des caractères corrompu	18
8.7	Synthèse	19
9	Écarts par rapport au cahier des charges	20
9.1	AWS API Gateway défini mais non utilisé en production	20
9.2	TTL de la mémoire conversationnelle	20
9.3	Couverture de tests	20
9.4	Documentation API	20
9.5	Extensions au-delà du périmètre initial	21
10	Bilan et compétences acquises	22
10.1	Compétences techniques	22
10.2	Compétences transversales	22
11	Conclusion et perspectives	23

Bibliographie	24
A Annexes	25
A.1 Arborescence du projet	25
A.2 Registre des risques (cahier des charges)	25
A.3 Liste consolidée des captures d’écran et schémas à ajouter	25

Table des figures

3.1 Planning prévisionnel du stage — 4 sprints de 2 semaines	6
5.1 Architecture globale du système	9
5.2 Modèle de données — DynamoDB et OpenSearch Serverless	10
5.3 Pipeline ReAct implémenté avec LangGraph (<code>graph/nodes.py</code>)	11
7.1 Taux de succès par catégorie — dernière exécution (23 scénarios)	16

Liste des tableaux

3.1 Stack technologique fixée par le cahier des charges	5
4.1 Synthèse du backlog Jira par Epic	7
4.2 User Stories principales et critères d’acceptance résumés	8
5.1 Outils mis à disposition de l’agent	12
7.1 Répartition des tests unitaires par module	15
8.1 Synthèse des anomalies corrigées durant le stage	19
A.1 Registre des risques et mitigation (source : cahier des charges)	25

Liste des abréviations

API	Application Programming Interface
AWS	Amazon Web Services
AOSS	Amazon OpenSearch Serverless
CDC	Cahier des charges
CDK	Cloud Development Kit (infrastructure-as-code AWS)
CI/CD	Intégration continue / Déploiement continu
IAM	Identity and Access Management
JWT	JSON Web Token
k-NN	k plus proches voisins (recherche vectorielle)
LLM	Large Language Model (grand modèle de langage)
PoC	Proof of Concept (preuve de concept)
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting (patron de conception d'agent)
REST	Representational State Transfer
SSE	Server-Sent Events
TTFT	Time To First Token
TTL	Time To Live
UI	User Interface (interface utilisateur)
US	User Story

Chapitre 1

Introduction

L'intelligence artificielle générative a connu, au cours des dernières années, une transition progressive : d'un simple générateur de texte répondant à une invite unique, vers des *agents autonomes* capables de raisonner sur plusieurs étapes, de mobiliser des outils externes et de maintenir un contexte sur la durée. Cette évolution ouvre des perspectives concrètes pour les entreprises de conseil technologique, qui cherchent à démontrer une expertise opérationnelle sur ces architectures plutôt qu'une simple maîtrise théorique des modèles de fondation.

C'est dans ce contexte que s'inscrit le stage de deux mois réalisé au sein de **Smartovate Ltd**, entreprise de conseil cloud, dont le sujet est le suivant : concevoir, développer et déployer un **agent IA autonome** s'appuyant sur **AWS Bedrock** pour l'accès aux modèles de fondation (Claude d'Anthropic, Titan d'Amazon), et sur **LangChain/LangGraph** pour l'orchestration du raisonnement. L'objectif n'est pas seulement de produire une démonstration ponctuelle, mais un prototype fonctionnel, testé et déployé sur l'infrastructure AWS de l'entreprise, capable de servir de brique réutilisable pour de futurs projets clients.

1.1 Contexte et enjeux

Un agent IA autonome se distingue d'un simple assistant conversationnel par sa capacité à *planifier* une séquence d'actions, à *exécuter* ces actions via des outils externes (recherche documentaire, calcul, appels API), et à *observer* les résultats obtenus pour ajuster son raisonnement — le tout de façon itérative, jusqu'à produire une réponse finale. Ce comportement, formalisé sous le nom de patron **ReAct** (*Reasoning and Acting*) [1], introduit des défis spécifiques : détection des boucles infinies, gestion du budget d'itérations, mémorisation du contexte sur plusieurs tours de conversation, et sécurisation des outils mis à disposition du modèle.

1.2 Objectifs du stage

- Le cahier des charges remis en début de stage fixe quatre objectifs techniques principaux :
- concevoir un agent ReAct capable d'utiliser des outils pour accomplir des tâches complexes ;
 - implémenter une mémoire hybride, combinant contexte de conversation à court terme et base vectorielle à long terme ;
 - déployer l'agent en architecture serverless sur AWS pour une scalabilité automatique ;
 - évaluer l'agent sur un jeu de tâches représentatives à l'aide de métriques objectives (précision, latence, taux de succès).

Au fil du stage, un cinquième axe de travail s'est imposé, non initialement détaillé dans le cahier des charges mais couvert par ses critères d'évaluation (30 % de la note portant sur la « qualité technique » et la couverture de tests) : la **fiabilisation** du système déjà en place, à

travers l'écriture de tests automatisés, la correction d'anomalies fonctionnelles et le durcissement de la sécurité de l'infrastructure. Ce cinquième axe occupe une place importante dans ce rapport (chapitres 7 et 8), car il représente une part significative du travail réellement effectué durant la seconde moitié du stage.

1.3 Organisation du rapport

Ce rapport suit la progression logique du projet. Le chapitre 2 présente brièvement l'entreprise d'accueil. Le chapitre 3 synthétise le cahier des charges et son planning. Le chapitre 4 décrit la méthode de travail adoptée (Agile/Scrum, gestion via Jira). Le chapitre 5 détaille l'architecture technique retenue. Le chapitre 6 décrit l'implémentation, organisée par grand axe fonctionnel (Epic). Le chapitre 7 présente la stratégie de test et les résultats obtenus. Le chapitre 8 revient sur les principales difficultés rencontrées et leur résolution. Le chapitre 9 formule un bilan honnête des écarts entre le cahier des charges et le système livré. Le rapport se conclut par un bilan des compétences acquises et des perspectives d'évolution.

Chapitre 2

Présentation de l'entreprise

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Cette section doit être réécrite avec les informations réelles de Smartovate Ltd (historique, effectif, implantation, clients-types, organisation de l'équipe technique). Le texte ci-dessous n'est qu'une structure indicative reprenant les seuls éléments présents dans le cahier des charges.

2.1 Activité

Smartovate Ltd est une entreprise de conseil cloud, dont l'activité consiste à accompagner des clients dans la conception, le déploiement et l'exploitation de solutions hébergées sur des plateformes cloud publiques. Dans le cadre de sa stratégie d'innovation, l'entreprise a souhaité se doter d'une expertise démontrable sur les services d'IA générative d'AWS, afin de proposer à ses clients des solutions sécurisées et performantes s'appuyant sur ces technologies. Le présent stage s'inscrit directement dans cette démarche : le prototype produit est conçu comme un *composant réutilisable*, pouvant être adapté à de futurs besoins clients.

2.2 Organisation de l'équipe d'accueil

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Décrire ici l'équipe technique encadrante, son positionnement dans l'organigramme, et le rôle du maître de stage.

Chapitre 3

Cadrage du projet

3.1 Périmètre

Le cahier des charges (rédigé conjointement avec l’entreprise et remis le 28 juin 2026) délimite précisément le périmètre du stage.

Inclus

- Configuration de l’environnement AWS (IAM, Bedrock, API Gateway, Lambda) ;
- développement de l’agent IA avec LangChain (Python) ;
- intégration de modèles de fondation via AWS Bedrock ;
- implémentation de la mémoire conversationnelle ;
- création d’outils personnalisés pour l’agent (recherche documentaire, interrogation d’une base de données) ;
- interface utilisateur basique (Streamlit ou API REST).

Explicitement exclu

- Mise en production à grande échelle — le projet reste un prototype / preuve de concept ;
- développement d’une interface utilisateur complexe (front-end avancé) ;
- entraînement ou *fine-tuning* de modèles de fondation (seuls des modèles pré-entraînés sont utilisés).

3.2 Stack technologique visée

TABLE 3.1 – Stack technologique fixée par le cahier des charges

Technologie / service	Rôle
AWS Bedrock (Claude 3 Sonnet / Titan)	Accès managé aux modèles de fondation
LangChain v0.3+ / LangGraph	Orchestration de l’agent
Python 3.11+	Langage de développement
Amazon DynamoDB	Mémoire court terme, journalisation
AWS Lambda	Exécution serverless de l’agent
Amazon S3	Stockage des documents utilisateur
AWS API Gateway	Point d’entrée HTTP (cf. écart §9.1)
Streamlit	Interface de démonstration
Docker	Empaquetage de la fonction Lambda

3.3 Planning prévisionnel

Le stage, d'une durée de huit semaines (deux mois), a été découpé en quatre sprints de deux semaines chacun, calqués sur la structure du *backlog* Jira (chapitre 4).

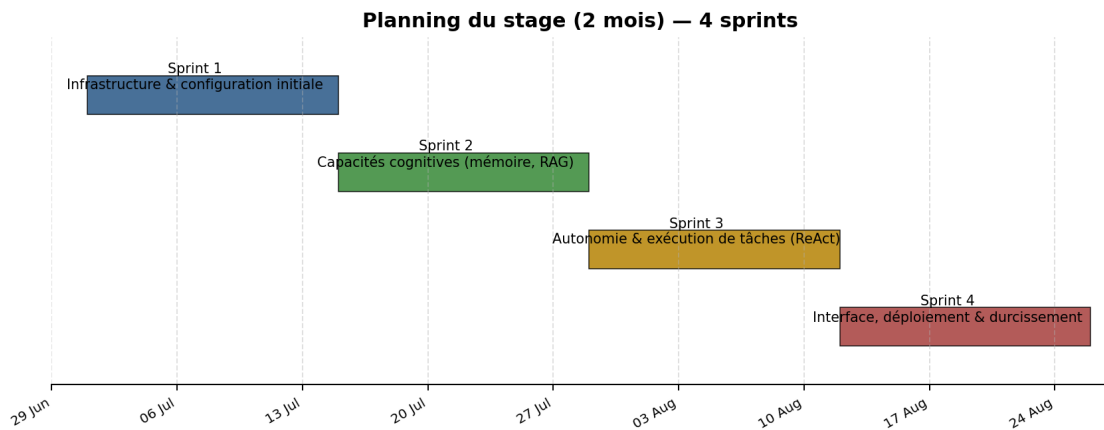


FIGURE 3.1 – Planning prévisionnel du stage — 4 sprints de 2 semaines

[Schéma généré automatiquement — fourni avec ce rapport]

Généré à partir des dates réelles du cahier des charges (`diagrams/planning_sprints.png`). Peut être remplacé par une capture d'écran du tableau Jira / Confluence si l'entreprise en dispose d'un plus détaillé.

Chapitre 4

Méthodologie de travail

4.1 Approche Agile et gestion de projet

Le projet a été mené selon une approche **Agile** / **Scrum** simplifiée, adaptée à un stage individuel : quatre sprints de deux semaines, chacun aligné sur un *Epic* du *backlog*, avec un point de synchronisation hebdomadaire avec le maître de stage.

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Décrire ici concrètement les rituels effectivement suivis (point hebdomadaire, revue de sprint, rétrospective éventuelle) et les outils de communication utilisés (Slack, Teams, email...).

4.2 Suivi via Jira

Le *backlog* a été structuré en 4 *Epics*, eux-mêmes décomposés en *Stories* puis en *Tasks* estimées en jours. Le tableau 4.1 synthétise cette hiérarchie.

TABLE 4.1 – Synthèse du backlog Jira par Epic

Clé	Epic	Stories	Tasks	Sprint
AGENT-EP-01	Setup AWS & Configuration Bedrock	1	3	Sprint 1
AGENT-EP-02	Développement de l'agent & outils	2	6	Sprint 2–3
AGENT-EP-03	Mémoire & persistance	2	4	Sprint 2
AGENT-EP-04	Déploiement serverless & API	1	3	Sprint 4

Le cahier des charges définit également quatre *User Stories* transverses (tableau 4.2), rédigées au format *persona* / *action* / *bénéfice* avec critères d'acceptance en notation *Given/When/Then*.

TABLE 4.2 – User Stories principales et critères d’acceptance résumés

ID	User Story	Critère d’acceptance clé
US-01	En tant qu’utilisateur, je veux poser une question multi-étapes, afin que l’agent la décompose et mobilise les bons outils.	≥ 2 outils différents exécutés, réponse en < 30 s, trace Thought/Action/Observation visible.
US-02	En tant qu’utilisateur, je veux reprendre une conversation interrompue, afin que l’agent se souvienne du contexte.	Accès aux 5 derniers échanges sans les renvoyer explicitement ; recherche en mémoire long terme fonctionnelle.
US-03	En tant que développeur, je veux ajouter un nouvel outil sans refactoriser l’existant.	Outil conforme à l’interface <code>Tool</code> utilisé automatiquement si pertinent ; appel journalisé dans DynamoDB.
US-04	En tant qu’opérateur, je veux consulter les logs en temps réel, afin de détecter boucles et erreurs.	Étapes ReAct visibles dans CloudWatch avec horodatage ; boucle détectée (même action $3\times$) $\Rightarrow$ arrêt propre.

[À COMPLÉTER PAR L’AUTEUR — capture d’écran non générable automatiquement]

Insérer ici une capture d’écran du tableau Jira (vue Kanban ou Backlog) montrant les Epics/Stories/Tasks réellement utilisés, ainsi qu’une capture de l’historique des commits Git montrant la fréquence des livraisons.

Chapitre 5

Conception de la solution

5.1 Vue d'ensemble de l'architecture

Le système repose sur une architecture **serverless** centrée autour d'une fonction **AWS Lambda** packagée en image Docker, exposée via une **Function URL** avec support du streaming de réponse (nécessaire au protocole SSE). La figure 5.1 présente les composants principaux et leurs interactions.

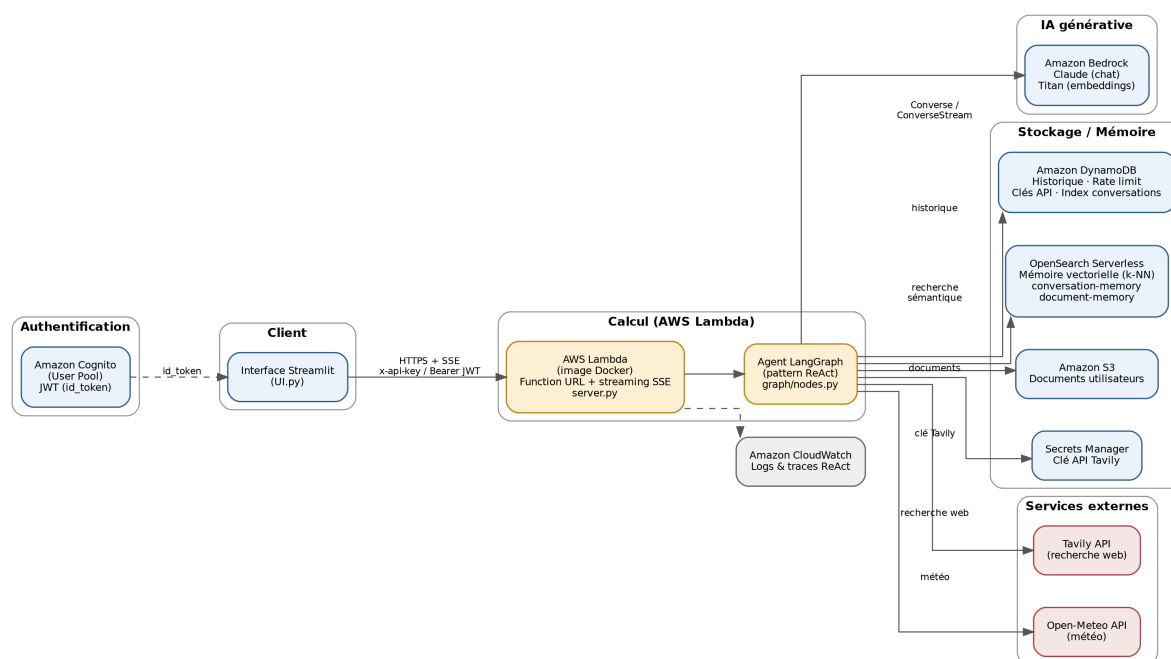


FIGURE 5.1 – Architecture globale du système

[Schéma généré automatiquement — fourni avec ce rapport]

Diagramme reconstitué à partir de l'analyse du code source et de l'infrastructure CDK réellement déployée (diagrams/architecture.png).

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Si l'entreprise dispose d'un schéma d'architecture officiel (dessiné sous draw.io / Lucidchart / AWS Architecture Icons en amont du projet), l'ajouter ici en complément ou en remplacement, et signaler explicitement les écarts éventuels entre le schéma prévu et l'architecture réellement implémentée (cf. §9).

5.2 Composants principaux

- **AWS Lambda (image Docker)** : héberge un serveur HTTP minimaliste (`server.py`, bibliothèque standard `http.server`) plutôt qu'un framework comme FastAPI, choix motivé par la nécessité de contrôler finement le streaming SSE des réponses.
- **Amazon Bedrock** : fournit l'accès managé au modèle de chat (Claude, via l'API *Converse/ConverseStream*) et au modèle d'embedding (Titan) utilisé pour la mémoire sémantique.
- **Amazon DynamoDB (4 tables)** : historique brut des conversations, compteurs de limitation de débit, clés API valides, index des conversations par utilisateur.
- **Amazon OpenSearch Serverless** : mémoire vectorielle (recherche par similarité k -NN) pour la mémoire sémantique long terme et la base de connaissance interne.
- **Amazon S3** : stockage des documents uploadés par les utilisateurs.
- **Amazon Cognito** : authentification des utilisateurs (JWT), au-delà du périmètre initial du cahier des charges (cf. §9).
- **AWS Secrets Manager** : stockage sécurisé de la clé API Tavily (recherche web).
- **Amazon CloudWatch** : centralisation des logs et des traces structurées de la boucle ReAct.
- **Interface Streamlit** : application cliente de démonstration, consommant l'API en streaming.

5.3 Modèle de données

Le système combine un stockage relationnel-clé (DynamoDB, pour l'historique brut et les métadonnées) et un stockage vectoriel (OpenSearch Serverless, pour la recherche sémantique). La figure 5.2 détaille le schéma logique de chaque table et index.

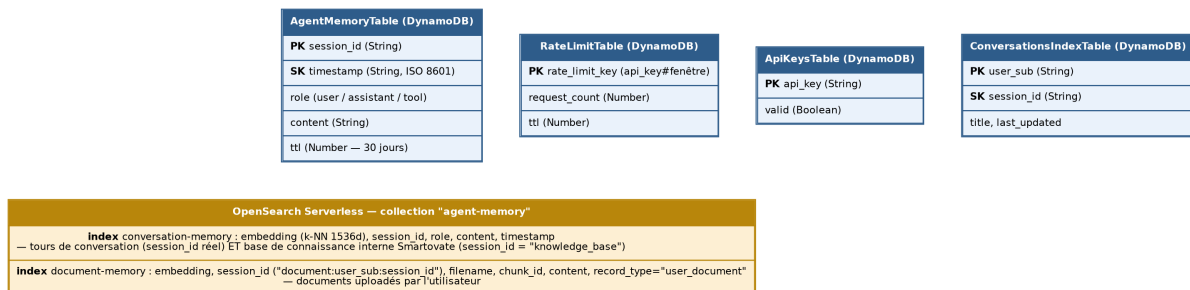


FIGURE 5.2 – Modèle de données — DynamoDB et OpenSearch Serverless

[Schéma généré automatiquement — fourni avec ce rapport]

Schéma extrait directement de l'implémentation (`config.py`, `opensearch_config.py`, `aws_ai_agent_stack.py`) — `diagrams/modele_donnees.png`.

Un choix de conception notable concerne le **stockage des embeddings** : ceux-ci ne sont jamais persistés dans DynamoDB (qui ne propose pas de recherche vectorielle native), mais exclusivement dans l'index OpenSearch dédié — décision qui a nécessité une correction en cours de stage (§8.2).

5.4 Le patron ReAct et son orchestration par LangGraph

L'agent est implémenté comme une machine à états **LangGraph**, alternant entre un nœud d'appel au modèle de langage (`call_model`) et un nœud d'exécution d'outil (`call_tools`), jusqu'à obtention d'une réponse finale. Deux garde-fous protègent le système contre les comportements pathologiques d'un agent autonome :

- un **plafond d'itérations** (10 par défaut, 20 au maximum), au-delà duquel l'agent s'arrête et explique la raison de l'arrêt à l'utilisateur ;
- une **détection de boucle** : si le même outil est appelé trois fois de suite avec des arguments strictement identiques, l'agent interrompt son exécution plutôt que de consommer indéfiniment des ressources (et du budget Bedrock).

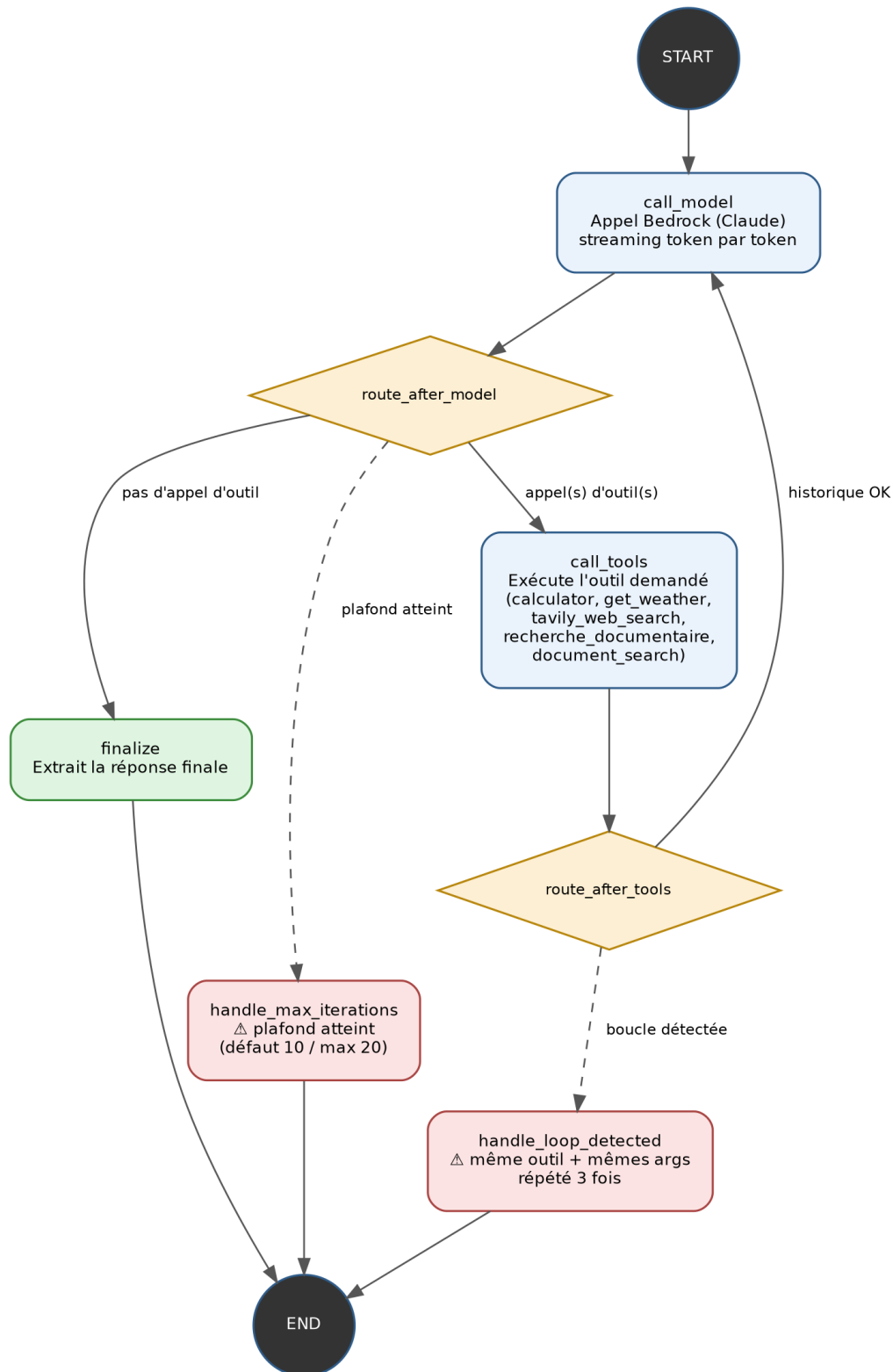


FIGURE 5.3 – Pipeline ReAct implémenté avec LangGraph (graph/nodes.py)

[Schéma généré automatiquement — fourni avec ce rapport]

Diagramme d'état reconstitué à partir du code source (`graph/nodes.py`, fonctions `route_after_model` et `route_after_tools`) — `diagrams/pipeline_react.png`.

Le code 5.1 illustre la logique de détection de boucle, volontairement simple (comparaison des trois derniers appels d'outils, indépendante de l'ordre des clés grâce à `json.dumps(..., sort_keys=True)`).

Listing 5.1 – Détection de boucle (`graph/nodes.py`)

```

1 LOOP_THRESHOLD = 3
2
3 def route_after_tools(state: AgentState) -> str:
4     history = state["tool_call_history"]
5     if len(history) < LOOP_THRESHOLD:
6         return "call_model"
7     last_n = history[-LOOP_THRESHOLD:]
8     signatures = {
9         json.dumps({"tool": c["tool"], "args": c["args"]}, sort_keys=True)
10        for c in last_n
11    }
12    if len(signatures) == 1:
13        return "loop_detected"
14    return "call_model"

```

5.5 Les cinq outils de l'agent

TABLE 5.1 – Outils mis à disposition de l'agent

Outil	Rôle
<code>calculator</code>	Évaluation d'expressions arithmétiques via <code>ast</code> , avec une liste blanche stricte de nœuds autorisés (aucun <code>eval()</code> brut).
<code>get_weather</code>	Météo en temps réel (géocodage + prévisions) via l'API Open-Meteo.
<code>tavily_web_search</code>	Recherche web via l'API Tavily, clé récupérée depuis Secrets Manager.
<code>recherche_documentaire</code>	RAG sur la base de connaissance interne de Smartovate (index <code>OpenSearch conversation-memory</code> , espace de noms <code>knowledge_base</code>).
<code>document_search</code>	RAG sur les documents uploadés par l'utilisateur (index <code>OpenSearch document-memory</code>), avec stratégie de nouvelles tentatives pour absorber la latence d'indexation <i>quasi</i> temps réel d'OpenSearch.

5.6 Sécurité

Trois mécanismes protègent l'API : **authentification Cognito** (JWT vérifié à chaque requête), **clé API** applicative, et **limitation de débit** (100 requêtes/minute par clé, avec stratégie *fail-open* : une panne de DynamoDB ne bloque jamais l'agent). Le principe du **moindre privilège** a été appliqué à l'ensemble des permissions IAM de la fonction Lambda (§8).

Chapitre 6

Réalisation

Ce chapitre détaille l'implémentation, organisée selon les quatre *Epics* du *backlog* (tableau 4.1).

6.1 Epic 1 — Setup AWS & Configuration Bedrock

Cette première étape a consisté à provisionner, sous forme de code (**AWS CDK**, Python), l'ensemble de l'infrastructure : rôle IAM de la fonction Lambda, activation de l'accès aux modèles Bedrock (Claude, Titan), tables DynamoDB, collection OpenSearch Serverless (avec ses trois politiques obligatoires : chiffrement, réseau, accès), bucket S3, secret Tavily. L'ensemble du stack tient dans un unique fichier `aws_ai_agent_stack.py`, ce qui facilite sa relecture mais a nécessité une vigilance particulière sur la portée des permissions accordées (§8.3).

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Capture d'écran de la console AWS Bedrock montrant les modèles Claude/Titan activés, et capture du résultat de `cdk deploy` en console.

6.2 Epic 2 — Développement de l'agent et de ses outils

Le cœur de l'agent repose sur **LangGraph** plutôt que sur l'**AgentExecutor** historique de LangChain mentionné dans le cahier des charges — ce choix, plus moderne, permet un contrôle explicite du graphe d'états (garde-fous, routage conditionnel) que ne permet pas l'API haut niveau de LangChain. Chaque outil est exposé au LLM via le décorateur `@tool` de `langchain_core` (équivalent fonctionnel à une classe héritant de **BaseTool**, avec une syntaxe plus concise). Le listing 6.1 illustre le traitement défensif appliqué à l'outil le plus sensible en matière de sécurité.

Listing 6.1 – Extrait de la liste blanche de `calculator` (`tools/calculator.py`)

```
1 _ALLOWED_NODES = (  
2     ast.Expression, ast.BinOp, ast.UnaryOp, ast.Constant,  
3     ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow, ast.Mod,  
4     ast.USub, ast.UAdd,  
5 )  
6  
7 def _safe_eval(expression: str) -> float:  
8     tree = ast.parse(expression, mode="eval")  
9     for node in ast.walk(tree):  
10         if not isinstance(node, _ALLOWED_NODES):  
11             raise ValueError(f"Expression non autorisée : {ast.dump(node)}")  
12     return eval(compile(tree, "<calculator>", "eval"))
```

6.3 Epic 3 — Mémoire et persistance

La mémoire court terme (fenêtre glissante des cinq derniers échanges, conforme à la spécification F3 du cahier des charges) est implémentée par une classe `DynamoDBChatMessageHistory` maison, plutôt que par la classe `ConversationBufferWindowMemory` de LangChain explicitement mentionnée dans le cahier des charges — celle-ci étant désormais dépréciée par l'éditeur au profit du protocole `BaseChatMessageHistory`, choix documenté dans le `README.md` du projet.

La mémoire long terme s'appuie sur une recherche de similarité (*top-k=3*) dans l'index vectoriel OpenSearch, déclenchée avant chaque réponse de l'agent pour retrouver le contexte le plus pertinent parmi les échanges passés.

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Capture d'écran de la console DynamoDB (tables créées, exemple d'item) et de la console OpenSearch Serverless (collection `agent-memory`, indices créés).

6.4 Epic 4 — Déploiement serverless et API

L'agent est empaqueté en image Docker et déployé sur AWS Lambda, exposé via une **Function URL** avec mode d'invocation `RESPONSE_STREAM`, permettant l'envoi de la réponse au fil de sa génération par le modèle (protocole SSE). L'interface Streamlit consomme ce flux pour afficher la réponse token par token, avec un indicateur de chargement pendant la phase de raisonnement.

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Capture d'écran de l'interface Streamlit en fonctionnement (conversation en cours, réponse en cours de streaming), et capture de la console Lambda montrant la fonction déployée et sa Function URL.

Chapitre 7

Tests et validation

La stratégie de validation du système repose sur deux niveaux complémentaires et volontairement distincts : des **tests unitaires**, qui vérifient la logique du code indépendamment de tout service externe réel, et un **harnais d'évaluation de bout en bout**, qui interroge le système réellement déployé.

7.1 Tests unitaires

161 tests ont été écrits avec `pytest`, répartis en 11 fichiers et couvrant l'ensemble des composants critiques (tableau 7.1). Aucun appel réseau ou AWS réel n'est effectué : les dépendances externes (Bedrock, OpenSearch, DynamoDB, Secrets Manager, Open-Meteo, Tavily) sont systématiquement simulées (`unittest.mock`), ce qui garantit une exécution rapide (moins de 20 secondes pour la suite complète) et déterministe.

TABLE 7.1 – Répartition des tests unitaires par module

Fichier de test	Nb tests	Couverture du module
<code>test_calculator.py</code>	20	100 %
<code>test_weather.py</code>	9	100 %
<code>test_tavily_web_search.py</code>	12	98 %
<code>test_search.py</code> (RAG interne)	8	100 %
<code>test_document_search.py</code>	14	86 %
<code>test_graph_nodes.py</code> (cœur ReAct)	23	85 %
<code>test_memory_service.py</code>	16	87 %
<code>test_rate_limit_service.py</code>	11	100 %
<code>test_utils.py</code>	12	100 %
<code>test_server_stream.py</code>	9	—
<code>test_evaluation_harness.py</code>	27	100 %
Total	161	71 % (ensemble du dossier lambda/)

La couverture globale (71 %) reste en-deçà de l'objectif de 80 % fixé par le cahier des charges ; l'écart est concentré sur les modules d'authentification et le point d'entrée HTTP complet (`server.py`), plus coûteux à tester unitairement du fait de leurs nombreuses dépendances externes imbriquées (cf. §9).

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Capture d'écran de l'exécution de `pytest --cov` en local montrant le résumé de couverture ligne par ligne.

7.2 Harnais d'évaluation de bout en bout

Un second niveau de validation, spécifiquement demandé par le cahier des charges (livrable « Rapport d'évaluation »), interroge le système **réellement déployé** sur 23 scénarios représentatifs : culture générale (aucun outil attendu), calculatrice, météo, recherche web, RAG interne, enchaînement de plusieurs outils, mémoire conversationnelle sur deux tours liés, respect du périmètre du *system prompt*, et robustesse (entrée invalide, tentative d'injection). Chaque scénario dispose d'un critère de succès automatisable (mots-clés attendus, outil déclenché ou non, rejet HTTP propre).

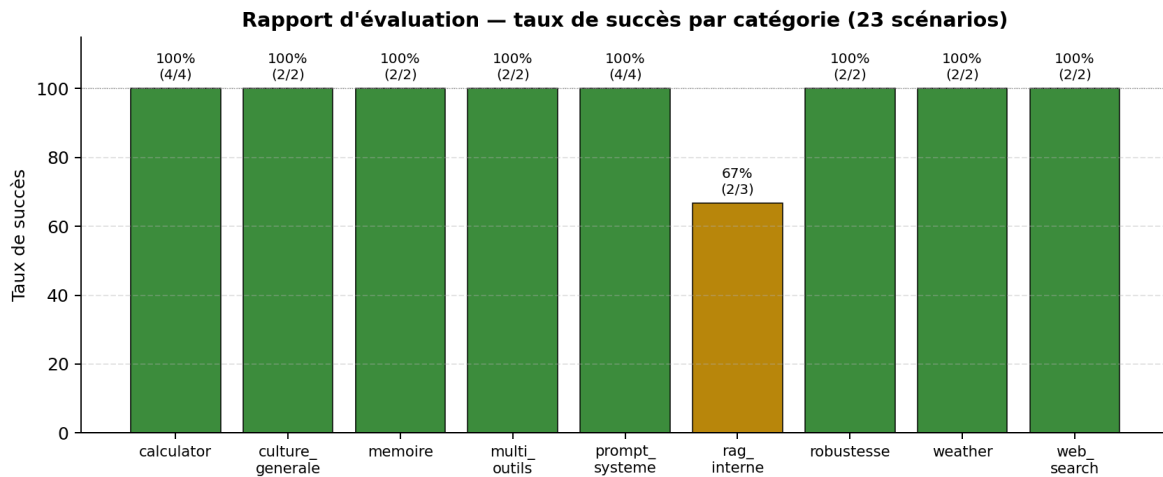


FIGURE 7.1 – Taux de succès par catégorie — dernière exécution (23 scénarios)

[Schéma généré automatiquement — fourni avec ce rapport]

Généré à partir du fichier `rapport_evaluation.csv` produit par le harnais (`diagrams/resultats_evaluation.png`). À régénérer avec les données de la dernière exécution avant remise finale du rapport.

Le taux de succès global atteint **95,7 %** (22/23), pour une latence moyenne de 8,1 secondes par requête (médiane 8,3 s, P95 à 11,7 s) — un temps de réponse cohérent avec un usage interactif, d'autant que le temps avant le premier caractère affiché (*Time To First Token*) reste proche de 2,9 secondes en médiane, ce qui donne à l'utilisateur une sensation de réactivité correcte malgré une réponse complète plus longue à produire.

Le seul scénario en échec (**rag-02**, catégorie « RAG interne ») est **informatif plutôt que révélateur d'un bug** : l'agent interroge correctement sa base de connaissance, mais celle-ci ne contient pas l'information demandée (les modèles de fondation utilisés par Smartovate) ; l'agent répond alors honnêtement qu'il ne sait pas, plutôt que d'halluciner une réponse — comportement souhaitable du point de vue de la fiabilité, même s'il compte comme un échec au sens strict du critère de test.

Chapitre 8

Difficultés rencontrées et solutions

Cette section documente les principales anomalies identifiées et corrigées en cours de stage. Chacune a été découverte par une combinaison de lecture de code, de tests ciblés et, pour certaines, d'exécution réelle via le harnais d'évaluation — une démarche méthodique qui constitue en elle-même un apprentissage important de ce stage.

8.1 Reconnexion silencieuse défailante

Symptôme : après un rafraîchissement de la page Streamlit, l'utilisateur était systématiquement renvoyé à l'écran de connexion, malgré la présence d'un jeton de rafraîchissement valide en cookie.

Cause : la bibliothèque de gestion de cookies utilisée renvoie un dictionnaire vide (`{}`) par défaut tant que le composant JavaScript sous-jacent n'a pas eu le temps de répondre — et non `None`, comme le supposait le code de vérification (`if cookies is None: st.stop()`). Ce test ne se déclenchait donc jamais, et le code concluait à tort à l'absence de cookie dès le premier cycle de rendu.

Correction : appel direct au composant sous-jacent avec un `default=None` explicite, restaurant la logique d'attente initialement prévue.

8.2 Mémoire sémantique jamais indexée

Symptôme : la recherche sémantique sur l'historique des conversations ne retournait jamais de résultat pertinent.

Cause : les *embeddings* étaient bien calculés à chaque tour de conversation, mais uniquement transmis à la fonction de sauvegarde DynamoDB — jamais à `vector_store.save_document()`, la fonction responsable de l'indexation réelle dans OpenSearch. Le chemin de production (`server.py`) désactivait même explicitement ce paramètre (`user_embedding=None`).

Correction : DynamoDB ne conserve désormais que le texte brut (il n'offre pas de recherche vectorielle native), et un appel explicite à `vector_store.save_document()` a été ajouté après chaque tour de conversation, pour les messages utilisateur et assistant.

8.3 Permissions IAM excessivement larges

Symptôme : la politique IAM de la fonction Lambda accordait un accès complet (`dynamodb:*`, actions Bedrock, actions OpenSearch) sur **toutes** les ressources du compte (`Resource: "*"`), avec un commentaire du code indiquant explicitement « à restreindre en production » — en contradiction directe avec le critère d'acceptance de la toute première tâche du projet (« les rôles et politiques IAM sont créés avec le principe de moindre privilège »).

Correction : chaque table DynamoDB dispose désormais d'une permission scoping à son seul ARN (via les méthodes `grant_read_write_data` du CDK) ; la permission `OpenSearch` (`aoss:APIAccessAll`, seule action IAM que propose AWS pour ce service) est restreinte à l'ARN de l'unique collection utilisée ; les permissions Bedrock sont limitées aux familles de modèles réellement invoquées. Validation effectuée par synthèse du *template* CloudFormation généré et inspection automatisée de l'absence de tout `Resource` : `"*"`.

8.4 Secret d'API en clair dans le terminal

Symptôme : la clé de l'API Tavily devait être ressaisie dans le terminal (`$env:TAVILY_API_KEY=...`) avant chaque déploiement CDK.

Correction : la clé est désormais stockée une seule fois dans **AWS Secrets Manager** ; le *stack* CDK référence le secret par son nom (jamais sa valeur), et la fonction Lambda le récupère elle-même au démarrage, avec mise en cache mémoire pour la durée de vie du conteneur d'exécution.

8.5 Garde-fous de sécurité invisibles côté utilisateur

Symptôme : lorsque l'un des deux garde-fous de la boucle ReAct se déclenchait (plafond d'itérations atteint, ou boucle détectée), l'utilisateur ne recevait **aucun message** — la bulle de réponse restait vide dans l'interface, sans aucune explication.

Cause : le message d'explication généré par ces deux nœuds terminaux n'est jamais produit par le modèle de langage (il s'agit d'une chaîne de caractères Python fixe) et n'est donc jamais diffusé via les événements de *streaming* token par token qui alimentent normalement l'affichage côté client ; le mécanisme de récupération prévu dans le code testait par erreur le nom du mauvais nœud du graphe.

Correction : émission explicite d'un événement de type « token » contenant le message du garde-fou dès qu'un de ces deux nœuds terminaux produit une réponse, sans dupliquer l'affichage sur le chemin de succès normal.

8.6 Encodage des caractères corrompu

Symptôme : les caractères accentués et les émojis apparaissaient corrompus dans les réponses de l'agent (par exemple « é » affiché « Ã© »), constaté concrètement dans les réponses collectées par le harnais d'évaluation.

Cause : l'en-tête `HTTP Content-Type` du flux SSE n'annonçait pas explicitement l'encodage UTF-8 utilisé pour encoder le corps de la réponse ; un client HTTP sans indication d'encodage retombe par défaut sur Latin-1, provoquant une réinterprétation erronée, octet par octet, de tout caractère non-ASCII.

Correction : ajout explicite de `charset=utf-8` à l'en-tête `Content-Type` des réponses HTTP et du flux SSE.

8.7 Synthèse

TABLE 8.1 – Synthèse des anomalies corrigées durant le stage

Anomalie	Impact	Sévérité
Reconnexion silencieuse cassée	Ré-authentification systématique	Élevée
Embeddings jamais indexés	Mémoire sémantique inopérante	Élevée
IAM Resource: "*"	Surface d'attaque excessive	Élevée
Secret Tavily en clair	Friction opérationnelle, risque de fuite	Moyenne
Garde-fous invisibles	Réponse vide sans explication	Élevée
Encodage UTF-8 cassé	Caractères corrompus	Moyenne

Chacune de ces corrections est verrouillée par au moins un test de non-régression, garantissant qu'elle ne pourra être réintroduite sans faire échouer immédiatement la suite de tests.

Chapitre 9

Écarts par rapport au cahier des charges

Par souci de transparence, ce chapitre recense les écarts assumés ou constatés entre les spécifications du cahier des charges et le système effectivement livré.

9.1 AWS API Gateway défini mais non utilisé en production

Le cahier des charges prescrit explicitement AWS API Gateway comme point d'entrée HTTP. Une ressource `RestApi` est bien définie dans le *stack* CDK (clé API, plan d'utilisation, limitation de débit), mais l'interface Streamlit interroge en réalité directement la **Function URL** de la fonction Lambda. Ce choix découle d'une contrainte technique réelle : le streaming de réponse token par token (SSE) n'est pas nativement supporté par l'API Gateway REST classique, alors qu'il l'est par les Function URLs Lambda en mode `RESPONSE_STREAM`. La ressource API Gateway laissée dans le *stack* constitue toutefois une zone d'ombre à clarifier (suppression ou finalisation) avant toute mise en production ultérieure.

9.2 TTL de la mémoire conversationnelle

Le cahier des charges spécifie un TTL de 7 jours pour la mémoire utilisateur en DynamoDB ; l'implémentation actuelle utilise 30 jours.

9.3 Couverture de tests

L'objectif de 80 % de couverture n'est atteint que sur les modules critiques (outils, cœur ReAct, mémoire, limitation de débit — 85 à 100 %) ; la couverture globale du dossier `lambda/` est de 71 %, les modules d'authentification et le point d'entrée HTTP complet restant partiellement non couverts.

9.4 Documentation API

Le cahier des charges et le *backlog* Jira mentionnent une documentation Swagger/OpenAPI de l'API. Le passage à un serveur HTTP natif Python (plutôt que FastAPI, pour les raisons de streaming évoquées en §9.1) a fait perdre la génération automatique de cette documentation ; aucune spécification OpenAPI écrite à la main ne la remplace à ce jour.

9.5 Extensions au-delà du périmètre initial

À l'inverse, certains éléments livrés dépassent le périmètre du cahier des charges : authentification multi-utilisateur via Amazon Cognito (non demandée explicitement), isolement strict des documents et conversations par utilisateur, et harnais d'évaluation automatisé réutilisable (au-delà d'un simple rapport ponctuel).

Chapitre 10

Bilan et compétences acquises

10.1 Compétences techniques

- Conception et exploitation d'un agent conversationnel basé sur le patron ReAct, avec LangChain / LangGraph ;
- utilisation opérationnelle d'AWS Bedrock (modèles de chat et d'embedding, API *Converse*) ;
- infrastructure-as-code avec AWS CDK (Python), incluant la configuration d'OpenSearch Serverless et la gestion fine des permissions IAM ;
- conception d'une architecture serverless avec streaming HTTP (Server-Sent Events) ;
- méthodologie de test logicielle : tests unitaires avec simulation systématique des dépendances externes, conception d'un harnais d'évaluation de bout en bout avec métriques objectives ;
- diagnostic et correction d'anomalies dans un système distribué (authentification, streaming, encodage, sécurité IAM).

10.2 Compétences transversales

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Compléter avec les compétences propres au vécu du stage : gestion du temps sur 8 semaines, communication avec le maître de stage, autonomie dans la priorisation des tâches, etc.

Chapitre 11

Conclusion et perspectives

Ce stage de deux mois a permis de livrer un agent IA autonome fonctionnel, répondant à l'essentiel des exigences du cahier des charges initial : architecture ReAct avec garde-fous, mémoire hybride court/long terme, cinq outils opérationnels, interface de démonstration en streaming, et déploiement effectif sur l'infrastructure AWS de Smartovate Ltd. Au-delà du strict périmètre fonctionnel, une part substantielle du travail a consisté à fiabiliser un système déjà partiellement construit : 161 tests unitaires, un harnais d'évaluation à 23 scénarios (95,7 % de réussite en conditions réelles), et la correction de six anomalies fonctionnelles et de sécurité significatives.

Les principales pistes d'amélioration identifiées à l'issue du stage sont : porter la couverture de tests à 80 % sur l'ensemble du code (notamment le serveur HTTP et l'authentification), clarifier le positionnement d'API Gateway dans l'architecture, documenter l'API au format OpenAPI, et mettre en place les mécanismes de maîtrise des coûts recommandés par le registre de risques du cahier des charges (budgets AWS, *Provisioned Concurrency*).

[À COMPLÉTER PAR L'AUTEUR — capture d'écran non générable automatiquement]

Conclure par une réflexion personnelle sur l'apport du stage au projet professionnel, et les suites envisagées (poursuite avec Smartovate, autres opportunités, etc.).

Bibliographie

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. *ReAct : Synergizing Reasoning and Acting in Language Models*. arXiv :2210.03629, 2022.
- [2] LangChain / LangGraph — Documentation officielle. <https://python.langchain.com> et <https://langchain-ai.github.io/langgraph/>
- [3] AWS. *Amazon Bedrock — User Guide*. <https://docs.aws.amazon.com/bedrock/>
- [4] AWS. *Amazon OpenSearch Serverless — Developer Guide*. <https://docs.aws.amazon.com/opensearch-service/>
- [5] AWS. *AWS Cloud Development Kit (CDK) — Developer Guide*. <https://docs.aws.amazon.com/cdk/>

Annexe A

Annexes

A.1 Arborescence du projet

Listing A.1 – Structure simplifiée du dépôt

```
1 aws-ai-agent/  
2 |-- UI.py # Interface Streamlit  
3 |-- aws_ai_agent/  
4 |   '-- aws_ai_agent_stack.py # Infrastructure AWS CDK  
5 |-- lambda/  
6 |   |-- server.py # Serveur HTTP + streaming SSE  
7 |   |-- graph/  
8 |     |-- nodes.py # Noeuds et routage ReAct  
9 |     |-- state.py # Etat du graphe (AgentState)  
10 |     '-- workflow.py # Construction du graphe LangGraph  
11 |-- tools/ # 5 outils de l'agent  
12 |-- services/ # Memoire, rate limiting, auth, ...  
13 |-- evaluation/ # Harnais d'evaluation (23 scenarios)  
14 |-- tests/ # 161 tests unitaires (pytest)
```

A.2 Registre des risques (cahier des charges)

TABLE A.1 – Registre des risques et mitigation (source : cahier des charges)

ID	Risque	Probab.	Impact
R1	Hallucinations de l'agent sur des tâches factuelles	Haute	Haute
R2	Timeout Lambda (15 min) pour les agents lents	Moyenne	Haute
R3	Coûts Bedrock imprévus	Moyenne	Haute
R4	Mauvaise sélection d'outil par le LLM	Haute	Moyenne
R5	Démarrage à froid Lambda (>5 s de latence)	Moyenne	Basse

A.3 Liste consolidée des captures d'écran et schémas à ajouter

- Console AWS Bedrock — modèles activés (§6) ;
- résultat de `cdk deploy` en console (§6) ;
- console DynamoDB — tables et exemple d'item (§6) ;
- console OpenSearch Serverless — collection et index (§6) ;
- interface Streamlit en fonctionnement (§6) ;
- console Lambda — fonction et Function URL (§6) ;

- tableau Jira (Kanban / Backlog) (§4) ;
- historique des commits Git (§4) ;
- exécution de `pytest -cov` en local (§7) ;
- logo de l'établissement et logo Smartovate Ltd (page de garde) ;
- schéma d'architecture officiel de l'entreprise, le cas échéant (§5).